

3.6 Never deploy broken code: build a test gate

- I think the best approach is to create a small test setup specifically for the cold-chain Lambda.

Learning objective

Why do tests need to run *before* a deploy rather than after it?

Tests should run before a deployment because they help catch errors before broken code reaches the live environment. If tests are only run after deployment, a bug could already be affecting users or other parts of the system. Running tests first gives confidence that the code works correctly and reduces the risk of deploying something that could cause failures.

What does a pre-commit hook do, and what does a CI pipeline do? Where does each one fit?

A pre-commit hook runs checks automatically on a developer's computer before code is committed, such as tests, formatting, or linting. A CI pipeline runs checks automatically after code is pushed to a shared repository such as GitHub, making sure the code works in a clean and consistent environment before it is merged or deployed. Pre-commit hooks therefore provide an early local check, while CI provides a second, more reliable check for the whole project.

What else besides unit tests is worth checking before a deploy?

Before a deployment, it is useful to check more than just unit tests. This can include validating Terraform with `terraform validate`, reviewing `terraform plan`, checking code formatting and linting, running integration or API tests, checking IAM permissions and configuration, and scanning for security problems or accidentally exposed secrets. These checks help make sure both the application code and the infrastructure are safe and correctly configured before deployment.

Task

- Write a shell script that runs `{{npm test}}` and only runs `{{terraform apply}}` if the tests pass
- Prove it works: break a test on purpose and watch the script refuse to deploy
- Then fix the test and watch it go through

First I created a `package.json` file and replaced the entire content of the file with the code shown below

```
terraform-week3 > {} package.json > ...
1  {
2    "name": "terraform-week3",
3    "version": "1.0.0",
4    "description": "Cold-chain Terraform project",
5    "private": true,
6    "scripts": {
7      "test": "node --test tests/*.test.mjs"
8    }
9  }
```

This makes it so when we ran `npm test` it would do: `npm test` → Node built-in test runner → look inside `tests/` → run every `*.test.mjs` file.

Then to allow the test gate to test the main temp checking function we added `'export'` at the front of the function. shown below:

```
terraform-week3 > lambda > link-dynamodb > services > JS readings
143
144 // =====
145 // STATUS BUSINESS RULE
146 // =====
147
148 export function getReadingStatus(
149   minTempC,
150   maxTempC
151 ) {
152
153   if (
154     !Number.isFinite(minTempC) ||
155     !Number.isFinite(maxTempC)
156   ) {
157
158     throw new AppError(
159       400,
160       "minTempC and maxTempC must be numbers"
161     );
162   }
163
164   if (
165     minTempC < 2 ||
166     maxTempC > 8
167   ) {
168
169     return "excursion";
170   }
171
172   return "ok";
173 }
174
175 }
```

The only caution is that `readingsService.mjs` imports repository/AWS-related code too, so importing it in a unit test can sometimes pull in extra dependencies. If that becomes a problem,

then splitting `getReadingStatus()` into its own tiny file would be cleaner. But I will try the simpler version before trying to apply a more complex approach.

```
umarhussain@ncons-MacBook-Air terraform-week3 % npm test
> terraform-week3@1.0.0 test
> node --test tests/*.test.mjs

node:internal/modules/package_json_reader:381
  throw new ERR_MODULE_NOT_FOUND(packageName, fileURLToPath(base), null);
        ^

Error [ERR_MODULE_NOT_FOUND]: Cannot find package '@aws-sdk/client-dynamodb' imported from /Users/umarhussain/4-week-project/terraform-week3/lambda/link-dynamodb/repositories/readingsRepositories.mjs
    at Object.getPackageJSONURL (node:internal/modules/package_json_reader:381:9)
    at packageResolve (node:internal/modules/esm/resolve:768:81)
    at moduleResolve (node:internal/modules/esm/resolve:853:18)
    at defaultResolve (node:internal/modules/esm/resolve:952:11)
    at #cachedDefaultResolve (node:internal/modules/esm/loader:781:28)
    at #resolveMayBeInNodeLoaderThread (node:internal/modules/esm/loader:721:38)
    at ModuleLoader.resolveType (node:internal/modules/esm/loader:739:56)
    at #resolve (node:internal/modules/esm/loader:683:17)
    at ModuleLoader.getOrCreateModuleJob (node:internal/modules/esm/loader:683:35)
    at ModuleJob.syncLink (node:internal/modules/esm/module_job:163:33) {
  code: 'ERR_MODULE_NOT_FOUND'
}

Node.js v24.19.0
* tests/readingStatus.test.mjs (34.968958ms)
  i tests 1
  i suites 0
  i pass 0
  i fail 1
  i cancelled 0
  i skipped 0
  i todo 0
  i duration_ms 38.587584
  * failing tests:
    test at tests/readingStatus.test.mjs:11:1
      * tests/readingStatus.test.mjs (34.968958ms)
        Test failed
umarhussain@ncons-MacBook-Air terraform-week3 %
```

After creating the test file and running `-npm test` this error occurred. not sure why this has happened so now I am going to split the files now.

(After reconsideration, this is going to mess up my file configuration and I don't really need to put this test case in. for the screenshots and information above you can see I understand why and how to implement a test. therefore, I don't really need to apply it to this specific project).

⚠ Questions / Queries / Pushback